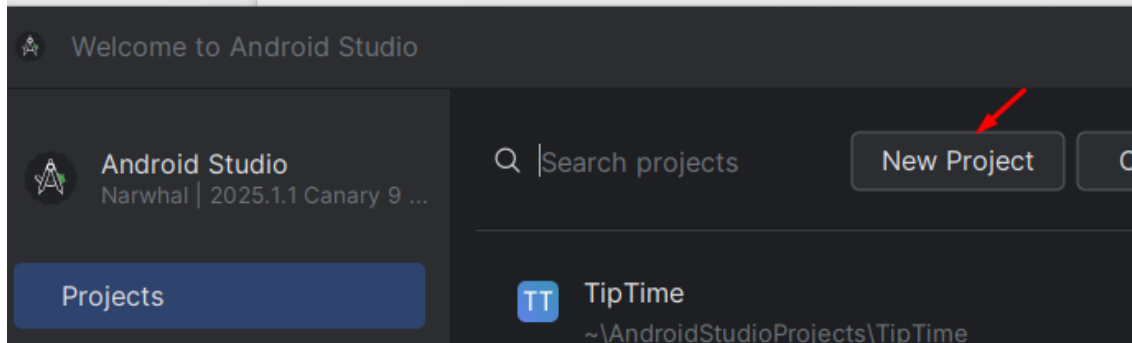


## Лабораторная работа №4. Создание приложения Dice Roller

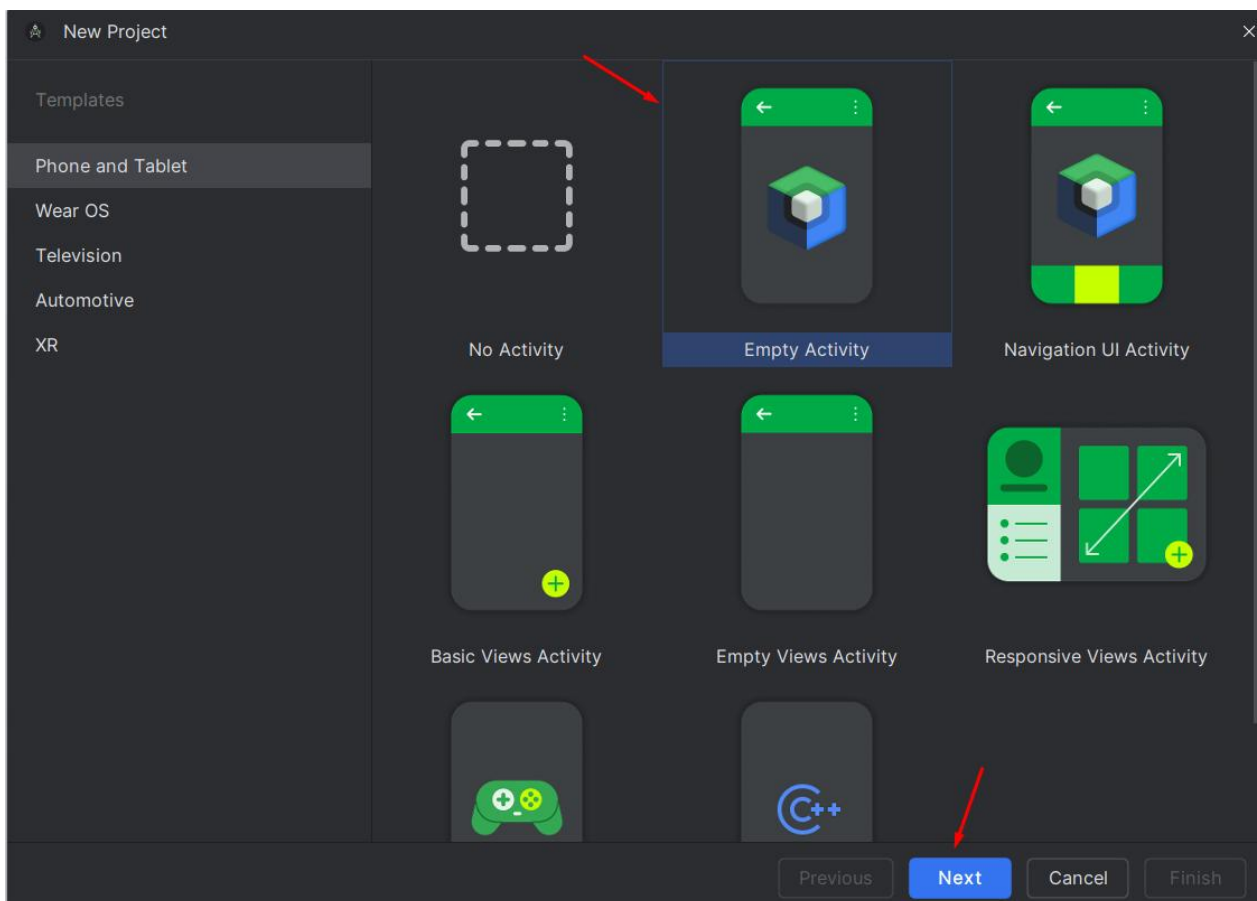
**Цель работы:** Создать интерактивное приложение **Dice Roller**, которое позволяет пользователям бросать игральные кости с помощью кнопки **Button**. Результат броска отображается на экране с использованием элемента **Image**.

### Шаг 1. Создание проекта

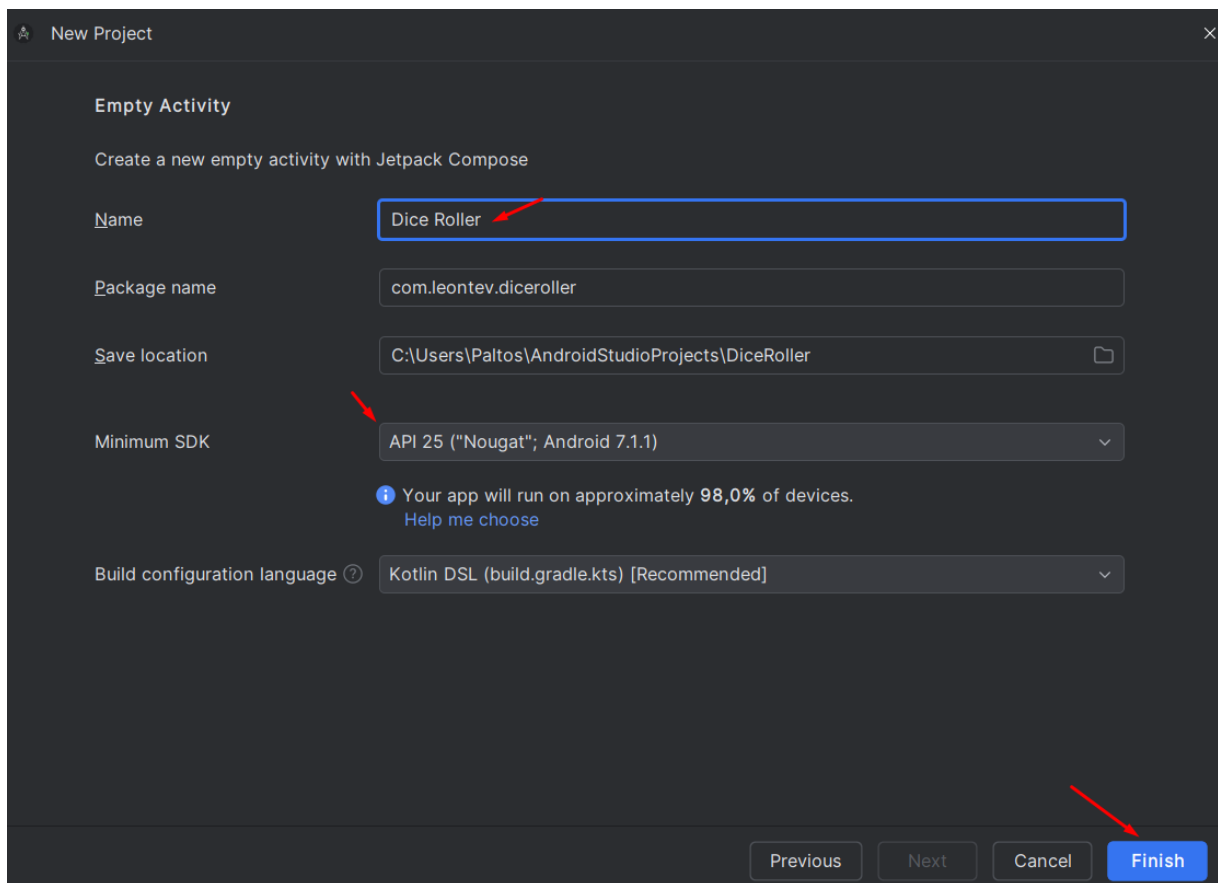
1. В **Android Studio** выберите **File > New > New Project**



2. В диалоговом окне «**New Project**» выберите «**Empty Activity**» и нажмите «**Next**»



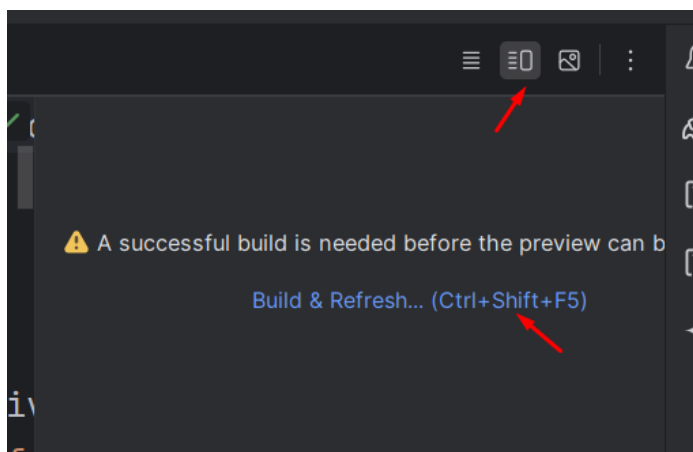
3. В поле **Name** введите **Dice Roller**. поле «**Minimum SDK**» выберите из меню минимальный уровень API 25, а затем нажмите «**Finish**»



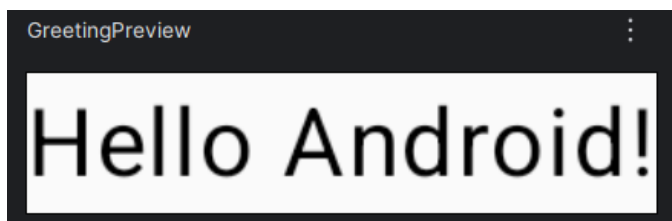
## Шаг 2. Создание макета

### 1. Предварительный просмотр проекта

- Нажмите «**Build & Refresh**» на панели «**Split**»



Теперь вы увидите предварительный просмотр на панели «**Design**»:



### 2. Реструктуризация кода

Необходимо изменить часть сгенерированного кода, чтобы он соответствовал тематике приложения для бросания игральных костей.

## Шаги:

1. Удалите функцию **GreetingPreview()**.

```
41  @Preview(showBackground = true)
42  @Composable
43  fun GreetingPreview() {
44      DiceRollerTheme {
45          Greeting(name = "Android")
46      }
47  }
```

2. Создайте функцию **DiceWithButtonAndImage()** с аннотацией **@Composable**.

```
41  @Composable
42  fun DiceWithButtonAndImage() {
43
44  }
```

Эта функция представляет компоненты пользовательского интерфейса (UI) макета и содержит логику нажатия кнопок и отображения изображений.

3. Удалите функцию **Greeting()**.

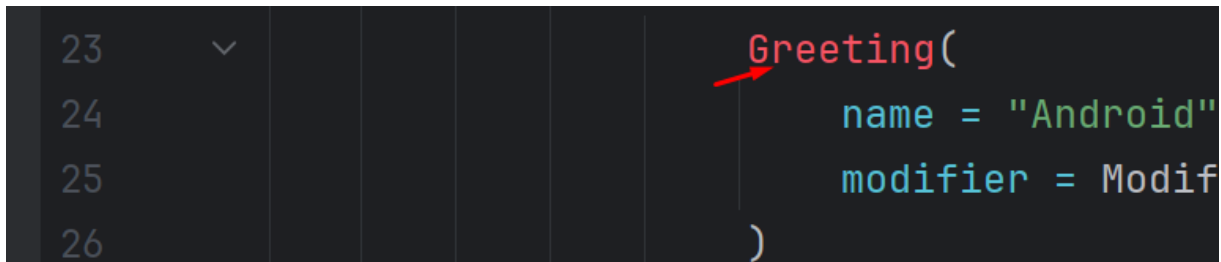
```
33  @Composable
34  fun Greeting(name: String, modifier: Modifier = Modifier) {
35      Text(
36          text = "Hello $name!",
37          modifier = modifier
38      )
39  }
```

4. Создайте функцию **DiceRollerApp()** с аннотациями **@Preview** и **@Composable**.

```
33  @Preview
34  @Composable
35  fun DiceRollerApp() {
36
37  }
38
```

### 3. Настройка функции `setContent`

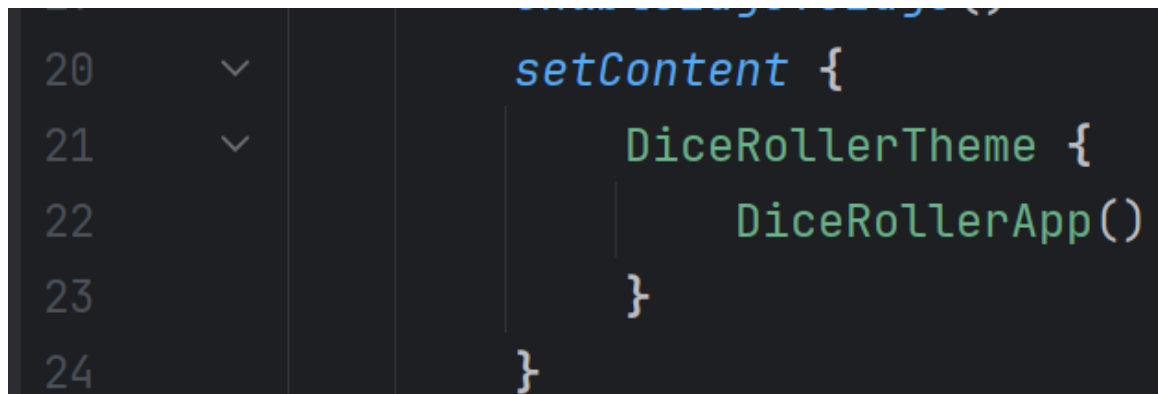
После удаления `Greeting()` вызов `Greeting("Android")` внутри лямбда-выражения `DiceRollerTheme()` подсвечен красным — компилятор не может найти ссылку на удалённую функцию.



```
23      Greeting(  
24          name = "Android"  
25          modifier = Modif  
26      )
```

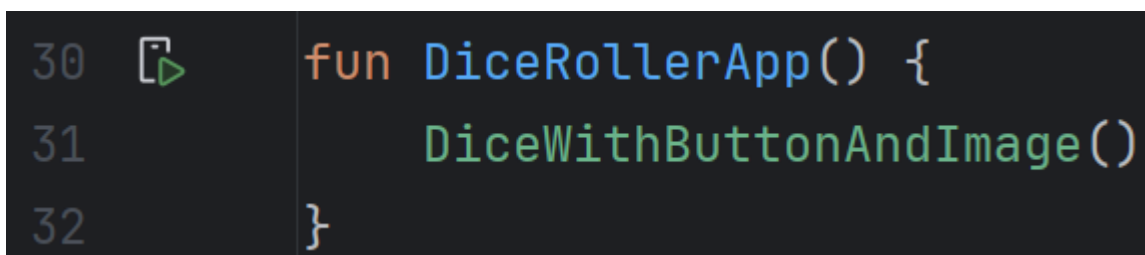
Чтобы исправить это:

1. Удалите весь код внутри лямбда-выражения `setContent {}` в методе `onCreate()`.
2. В теле `setContent {}` вызовите лямбду `DiceRollerTheme {}`, а внутри неё — функцию `DiceRollerApp()`.



```
20      setContent {  
21          DiceRollerTheme {  
22              DiceRollerApp()  
23          }  
24      }
```

3. В функции `DiceRollerApp()` вызовите функцию `DiceWithButtonAndImage()`.



```
30      fun DiceRollerApp() {  
31          DiceWithButtonAndImage()  
32      }
```

### 4. Работа с Modifier

**Compose** использует объект **Modifier**, который представляет собой набор элементов, украшающих или изменяющих поведение UI-компонентов. Он применяется для стилизации интерфейса приложения **Dice Roller**.

#### Добавление модификатора

1. Измените функцию `DiceWithButtonAndImage()`, чтобы она принимала аргумент **modifier** типа **Modifier** с параметром по умолчанию **Modifier**.

```

34     @Composable
35     fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
36
37     }

```

2. При вызове функции **DiceWithButtonAndImage()** передайте объект **Modifier** с нужными декорациями.

```

30  fun DiceRollerApp() {
31      DiceWithButtonAndImage(modifier = Modifier)
32  }

```

## 5. Применение методов Modifier

К объекту **Modifier**, передаваемому в **DiceWithButtonAndImage()**, добавьте следующие методы:

- **fillMaxSize()** — чтобы макет заполнил весь экран.
- **wrapContentSize(Alignment.Center)** — чтобы центрировать компоненты по вертикали и горизонтали.

```

36     @Composable
37     fun DiceWithButtonAndImage(
38         modifier: Modifier = Modifier
39         .fillMaxSize()
40         .wrapContentSize(Alignment.Center)
41     ) {

```

**! Если появляются ошибки импорта, добавьте:**

```

import androidx.compose.foundation.layout.wrapContentSize
import androidx.compose.ui.Alignment

```

## Шаг 3. Создание вертикальной компоновки

В **Jetpack Compose** вертикальные макеты создаются с помощью функции **Column()**. Функция **Column()** представляет собой компоновку макет, который размещает дочерние элементы **в вертикальной последовательности**.

### Порядок действий

1. В функции **DiceWithButtonAndImage()** добавьте вызов **Column()**.

2. Передайте **modifier**-аргумент из сигнатуры метода **DiceWithButtonAndImage()** в аргумент **modifier** функции **Column()**. Это обеспечит, что элементы внутри **Column()** будут следовать ограничениям и декорациям, заданным через объект **Modifier**.
3. Добавьте аргумент **horizontalAlignment** в **Column()** и присвойте ему значение **Alignment.CenterHorizontally**.

Это гарантирует, что все дочерние элементы будут **по центру экрана по горизонтали**.

```
37     @Composable
38     fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
39         Column(
40             modifier = modifier,
41             horizontalAlignment = Alignment.CenterHorizontally
42         ) {}
43     }
```

**! Если появляются ошибки импорта, добавьте:**

**import androidx.compose.foundation.layout.Column**

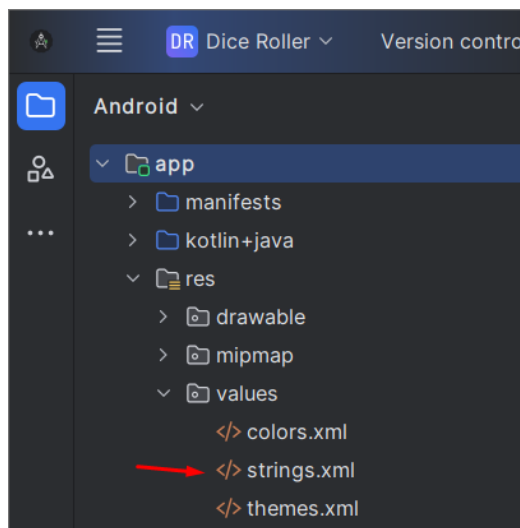
```
import androidx.compose.foundation.layout.Column
import androidx.compose.ui.Alignment
```

#### Шаг 4. Добавление кнопки

Теперь добавим кнопку, которая будет выполнять бросок кубика.

##### 1. Добавление строкового ресурса

Откройте файл: **res/values/strings.xml**



Добавьте новую строку:

```
<string name="roll">Roll</string>
```

```
... 1 <resources>
    2 <string name="app_name">Dice Roller</string>
    3 <string name="roll">Roll</string>
    4 </resources>
```

## 2. Добавление кнопки в макет

В теле лямбда-выражения **Column()** добавьте компокуемый объект **Button()**.

```
Button(onClick = { }) {
}
```

## 3. Добавление текста на кнопку

Внутри **Button()** добавьте элемент **Text()**, который будет отображать надпись **"Roll"**. Для получения значения строки используйте функцию **stringResource()**.

```
40 fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
41     Column(
42         modifier = modifier,
43         horizontalAlignment = Alignment.CenterHorizontally
44     ) {
45         Button(onClick = { }) {
46             Text(text = stringResource(id = R.string.roll))
47         }
48     }
49 }
```

**! Если появляются ошибки импорта, добавьте следующие пакеты:**

```
import androidx.compose.material3.Text
import androidx.compose.ui.res.stringResource
import androidx.compose.material3.Button
```

## Шаг 5. Добавление изображения

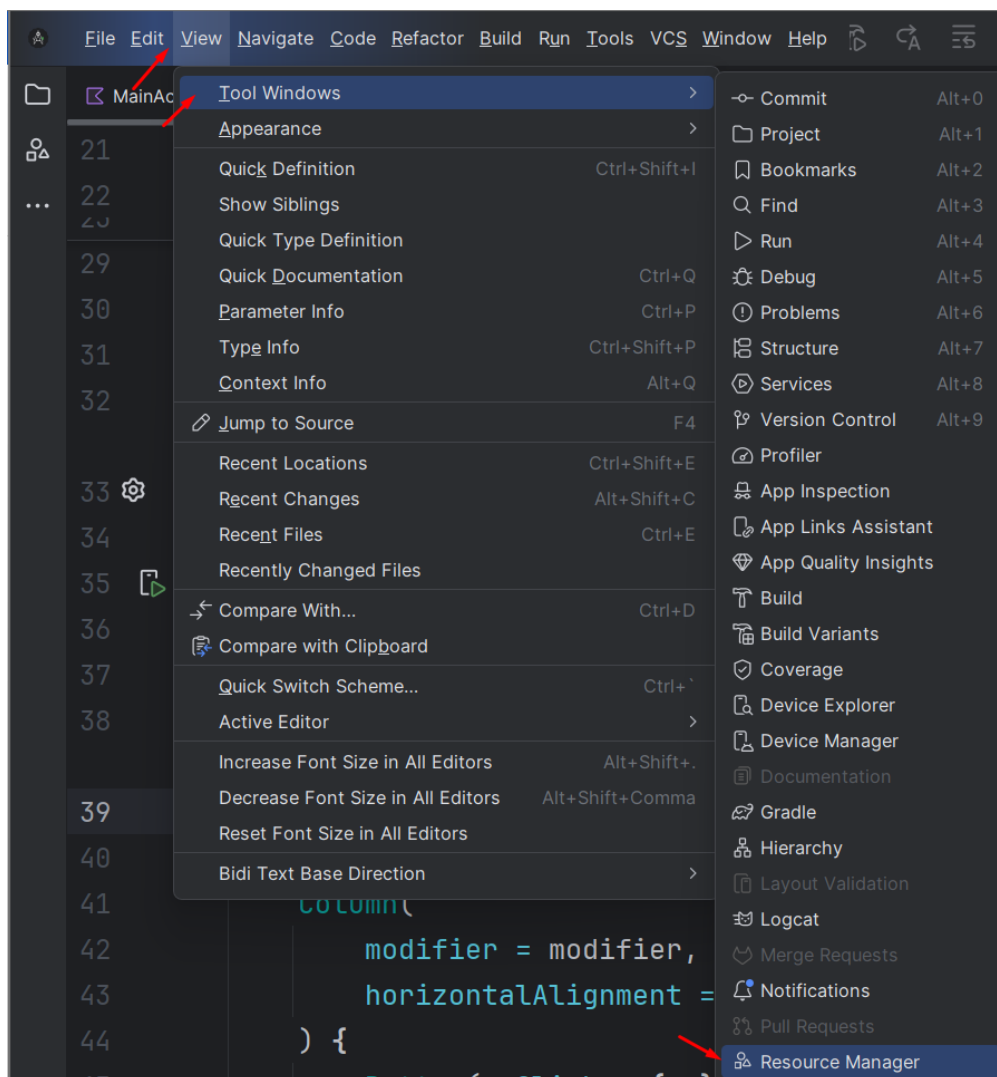
Ещё один важный компонент приложения — **изображение игровой кости**, которое будет отображать результат броска при нажатии кнопки «Бросить». Для отображения используется компокуемый объект **Image()**, которому требуется ресурс изображения. Поэтому сначала нужно **добавить изображения кубиков** в проект.

## 1. Загрузка изображений игральных костей

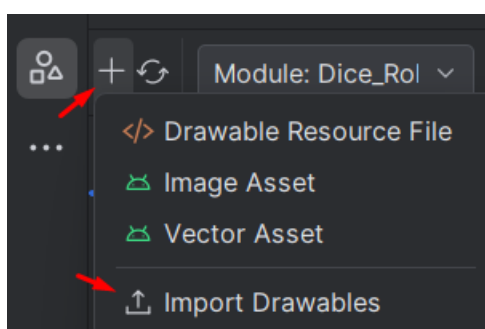
1. В приложении к лабораторной работе откройте предоставленный **ZIP-архив**.
2. Распакуйте архив — будет создана папка **dice\_images**, содержащая **шесть файлов изображений** кубиков со значениями от 1 до 6.

|            |                 |                              |      |
|------------|-----------------|------------------------------|------|
| dice_1.xml | 14.01.2022 2:21 | Microsoft Edge HTML Document | 4 KB |
| dice_2.xml | 14.01.2022 2:21 | Microsoft Edge HTML Document | 4 KB |
| dice_3.xml | 14.01.2022 2:21 | Microsoft Edge HTML Document | 4 KB |
| dice_4.xml | 14.01.2022 2:21 | Microsoft Edge HTML Document | 4 KB |
| dice_5.xml | 14.01.2022 2:21 | Microsoft Edge HTML Document | 4 KB |
| dice_6.xml | 14.01.2022 2:21 | Microsoft Edge HTML Document | 4 KB |

3. В **Android Studio** выберите **View > Tool Windows > Resource Manager**

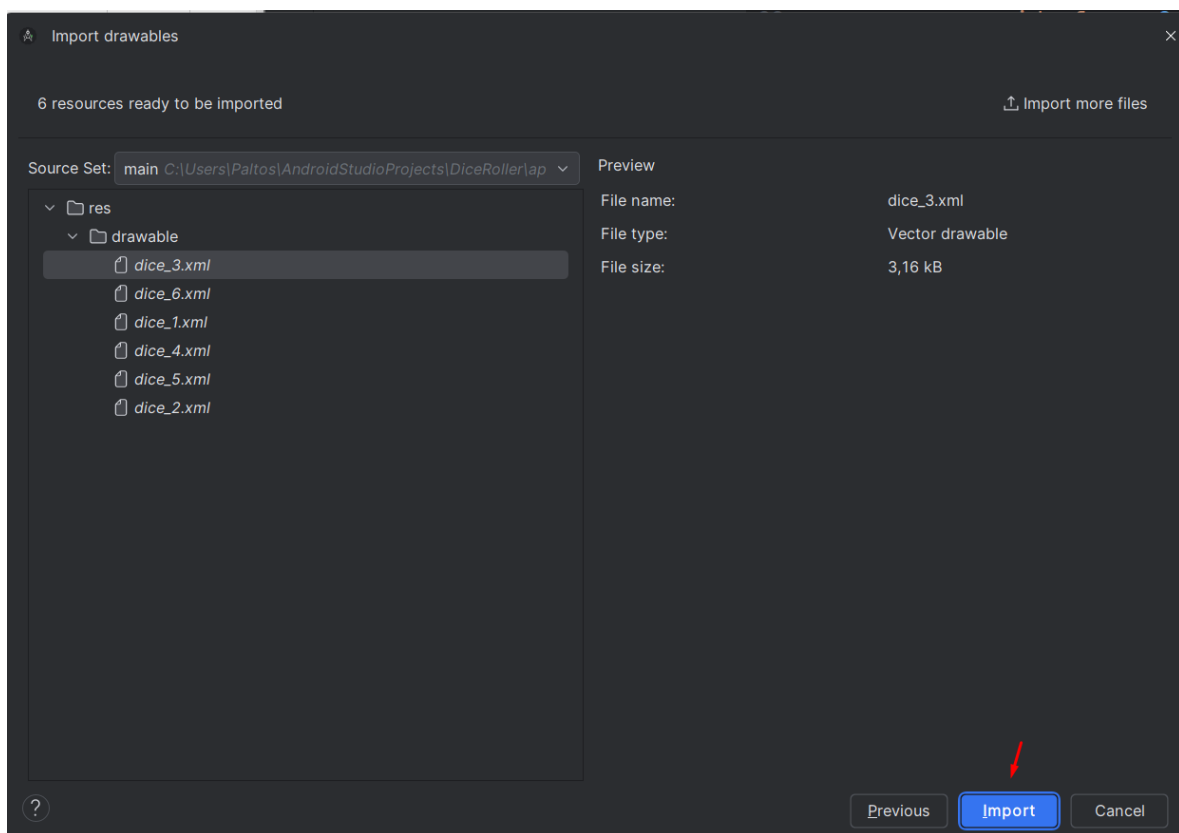
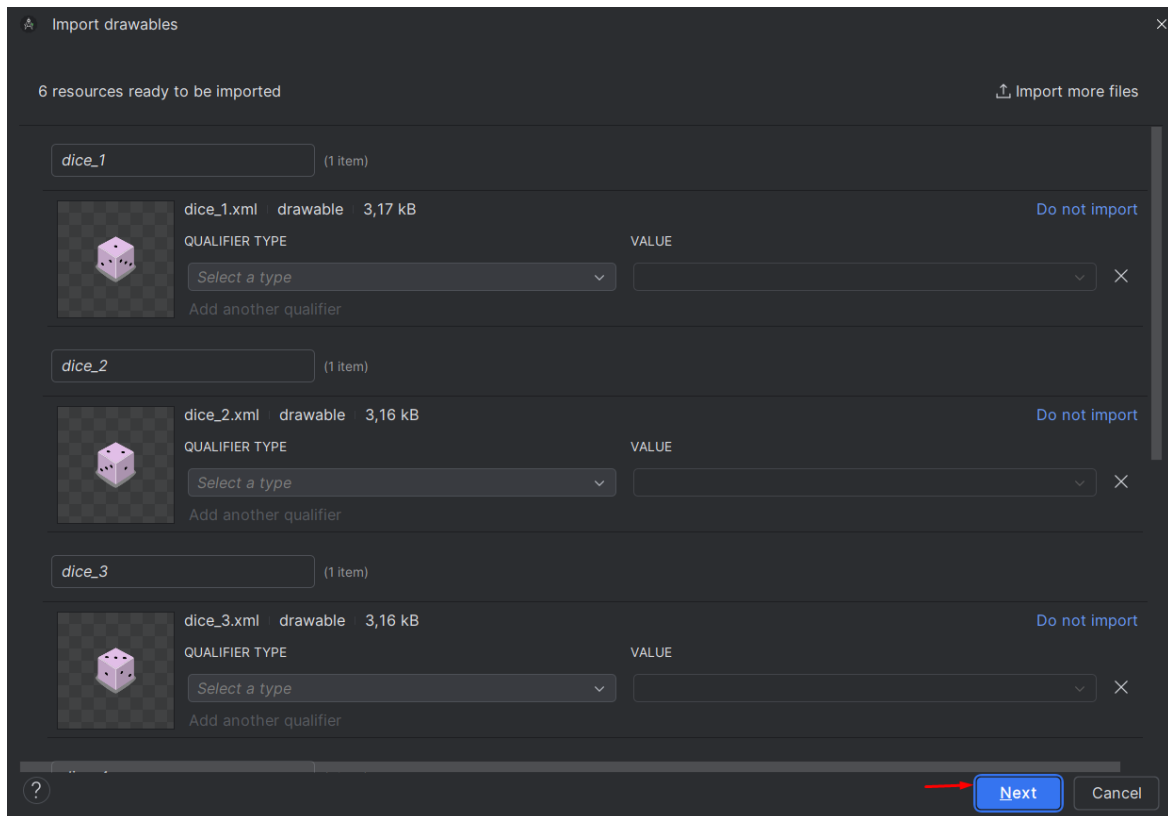


4. Нажмите **+ > Import Drawables**, чтобы открыть проводник файлов.

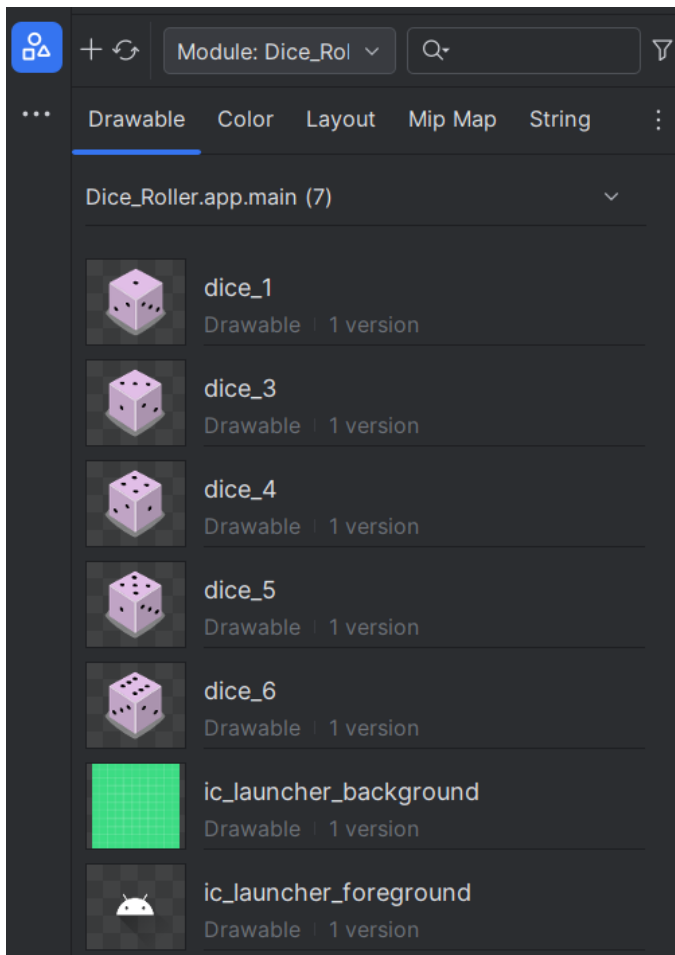




5. Найдите и выберите распакованную папку с изображениями кубиков.
6. Нажмите «Next», затем «Import » для подтверждения.



После импорта шесть изображений должны появиться на панели **Resource Manager**.



## 2. Доступ к изображениям из кода

Теперь вы можете ссылаться на изображения в коде Kotlin по их идентификаторам: `R.drawable.dice_1`; `R.drawable.dice_2`; `R.drawable.dice_3`; `R.drawable.dice_4`; `R.drawable.dice_5`; `R.drawable.dice_6`

## 3. Добавление Image composable

Изображение кубика должно располагаться над кнопкой «Бросить». В **Compose** элементы внутри **Column()** размещаются по порядку объявления: первый элемент — выше, последующие — ниже.

### Шаги:

1. Внутри **Column()** функции **DiceWithButtonAndImage()** создайте **Image()** перед **Button()**.

```
) {  
    Image()  
    Button(onClick = { }) {
```

Импортируйте для изображения:

```
import androidx.compose.foundation.Image
```

#### 4. Передача ресурса изображения

Добавьте аргумент `painter` и используйте функцию `painterResource()`, чтобы указать изображение. Пока используйте значение по умолчанию `R.drawable.dice_1`, чтобы код корректно компилировался для предпросмотра.

```
Image(  
    painter = painterResource(R.drawable.dice_1)  
)
```

Импортируйте:

```
painterResource(R.drawable.  
Unresolved reference 'painterResour  
< = { } Import function 'painterResource' A
```

#### 5. Добавление описания содержимого

Каждое изображение должно иметь **описание содержимого** (*content description*) для повышения доступности. Добавим в наше изображение:

```
painter = painterResource(id = R.drawable.dice_1),  
contentDescription = "1"
```

На данный момент описание "1" — это временная заглушка.

#### 6. Добавление отступа между Image и Button

Сейчас кнопка и изображение могут перекрываться. Чтобы добавить между ними пространство, используйте компонент `Spacer()`.

```
Image(  
    painter = painterResource(id = R.drawable.dice_1),  
    contentDescription = "1"  
)  
Spacer(modifier = Modifier.height(height = 16.dp))  
Button(onClick = { }) {
```

Необходимые импорты:

```
import androidx.compose.foundation.Image
```

```
import androidx.compose.foundation.layout.Spacer
```

```
import androidx.compose.foundation.layout.height
```

```
import androidx.compose.ui.res.painterResource
```

```
import androidx.compose.ui.unit.dp
```

После компиляции и нажатия «**Build & Refresh**» в панели предпросмотра вы должны увидеть макет, похожий на этот:



## Шаг 6. Построение логики броска кубика

Теперь, когда все необходимые элементы интерфейса созданы, нужно добавить логику броска кубика, чтобы при нажатии кнопки менялось изображение.

### 1. Сделайте кнопку интерактивной

В функции **DiceWithButtonAndImage()**, перед вызовом **Column()**, создайте переменную **result** и присвойте ей значение **1**:

```
fun DiceWithButtonAndImage() {  
    var result = 1
```

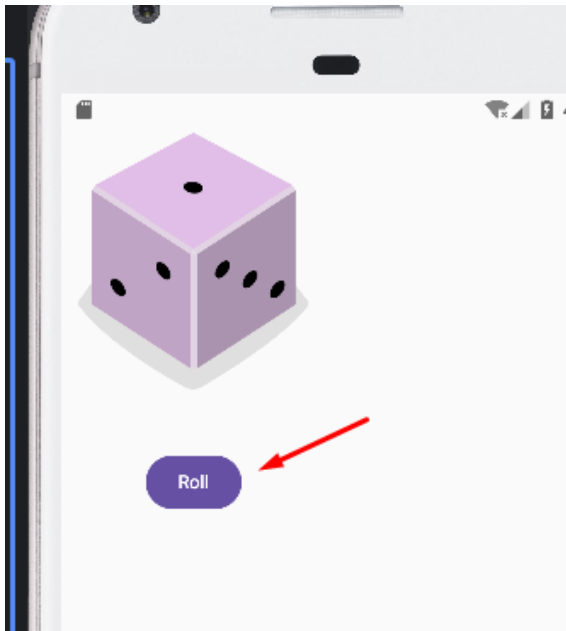
Посмотрите на компонуемый объект **Button()**. Параметр **onClick** принимает лямбда-выражение — участок кода, который выполняется при нажатии кнопки:

```
Button(onClick = {      }) {
```

Добавьте генерацию случайного числа от 1 до 6 при каждом клике. Для этого используйте метод `.random()` на диапазоне `1..6`:

```
Button(onClick = { result = (1 ≤ .. ≤ 6).random() }) {
```

Теперь кнопку можно нажимать (можно проверить в эмуляторе), но визуально ничего не меняется. Чтобы отображение обновлялось при каждом броске, нужно добавить **состояние (state)** в **Compose**.



## 2. Добавляем состояние через remember

По умолчанию компоуемые функции **не сохраняют значения** между пересозданиями интерфейса. Для хранения состояния используется `remember { mutableStateOf(...) }`.

Измените объявление переменной `result`:

```
var result by remember { mutableStateOf(value = 1) }
```

Теперь **Compose** будет отслеживать изменения переменной `result` и автоматически перерисовывать **UI** при её изменении.

Необходимые импорты:

```
import androidx.compose.runtime.remember
```

```
import androidx.compose.runtime.mutableStateOf
```

```
import androidx.compose.runtime.getValue
```

*import androidx.compose.runtime.setValue*

### 3. Определяем изображение для каждого результата

Создайте переменную **imageResource**, значение которой зависит от **result**.

Используйте выражение **when**:

```
var result by remember { mutableStateOf( value = 1) }  
val imageResource = when (result) {  
    1 -> R.drawable.dice_1  
    2 -> R.drawable.dice_2  
    3 -> R.drawable.dice_3  
    4 -> R.drawable.dice_4  
    5 -> R.drawable.dice_5  
    else -> R.drawable.dice_6  
}
```

### 4. Обновите Image

Теперь замените жёстко заданный ресурс **R.drawable.dice\_1** на переменную **imageResource**, а также передайте текущее значение **result** в качестве описания содержимого:

```
Image(  
    painter = painterResource(id = imageResource),  
    contentDescription = result.toString()  
)
```

После запуска приложения при каждом нажатии кнопки «**Roll**» (или «Бросить») изображение кубика будет случайным образом меняться от 1 до 6.

Ваше приложение **Dice Roller** теперь полноценно работает.

### Шаг 7. Добавляем выравнивание по центру экрана

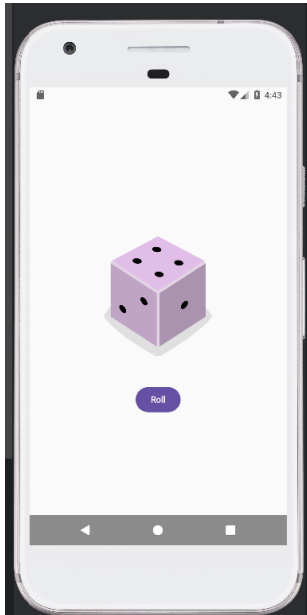
Сейчас наше приложение не по центру экрана. Добавим модификаторы для выравнивания всего содержимого по центру.

- **fillMaxSize()** — чтобы макет заполнил весь экран.
- **wrapContentSize(Alignment.Center)** — чтобы содержимое (**Column**) было выровнено по центру экрана.

Добавьте в функцию **DiceRollerApp()**:

```
fun DiceRollerApp() {
    DiceWithButtonAndImage(modifier = Modifier
        .fillMaxSize()
        .wrapContentSize(Alignment.Center))
}
```

Перезагрузите эмулятор, и проверьте что у вас кубик стал по центру экрана:



## Шаг 8. Добавление анимации броска кубика

Теперь, когда логика броска кубика работает, можно добавить **простую анимацию**, чтобы визуально показать «бросок» и сделать приложение более динамичным.

В этом шаге вы научитесь использовать **встроенные средства анимации Jetpack Compose** — без сторонних библиотек.

При каждом нажатии кнопки «Бросить» изображение кубика будет **плавно вращаться**. Для этого используется функция **animateFloatAsState()**, которая позволяет анимировать числовые значения (в нашем случае — угол вращения).

### Реализация анимации

В теле функции **DiceWithButtonAndImage()** создайте новую переменную **rotation**, которая будет хранить текущий угол вращения кубика:

```
var rotation by remember { mutableStateOf(value = 0f) }
```

Создайте анимируемое состояние, которое плавно изменяет значение **rotation** при каждом обновлении:

```
val animatedRotation by animateFloatAsState(
    targetValue = rotation,
    animationSpec = tween(durationMillis = 400),
    label = "dice_rotation"
)
```

Здесь:

- **targetValue** — целевое значение угла, к которому стремится анимация;
- **tween(durationMillis = 400)** — задаёт длительность анимации 0.4 секунды.

### Применение анимации к изображению

Добавьте модификатор **rotate()** к изображению кубика, чтобы применить эффект вращения:

```
Image(
    painter = painterResource(id = imageResource),
    contentDescription = result.toString(),
    modifier = Modifier
        .size(size = 150.dp)
        .rotate(degrees = animatedRotation)
)
```

### Изменение поведения кнопки

В обработчике **onClick** добавьте увеличение угла вращения на 360 градусов при каждом броске:

```
Button(
    onClick = {
        result = (1..6).random()
        rotation += 360f
    }
) {
```

Перезапустите и проверьте работу эмулятора.



Если изображение кубика кажется вам маленьким, то можете у модификатора **size** в **Image** изменить размер на больший:

```
.size( size = 350.dp)
```

## Шаг 9. Использование отладчика в Android Studio

Отладчик — это важный инструмент, позволяющий контролировать выполнение кода вашего Android-приложения и исправлять любые ошибки. Он позволяет указывать точки, в которых следует приостановить выполнение кода, и вручную взаимодействовать с переменными, методами и другими аспектами кода.

Запустить отладчик вместе с приложением можно двумя способами:

- Подключите отладчик к существующему процессу приложения, работающему на устройстве или эмуляторе.
- Запустите приложение с помощью отладчика.

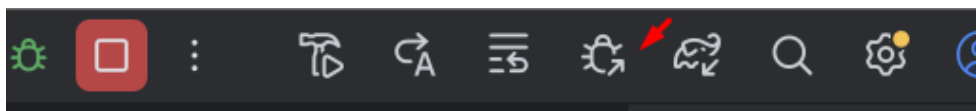
Оба способа в определённой степени достигают одного и того же. Освоив оба способа, вы сможете выбрать тот, который вам больше нравится или который необходим.

Присоедините отладчик к процессу приложения

Если ваше приложение уже запущено, вы можете прикрепить к нему отладчик.

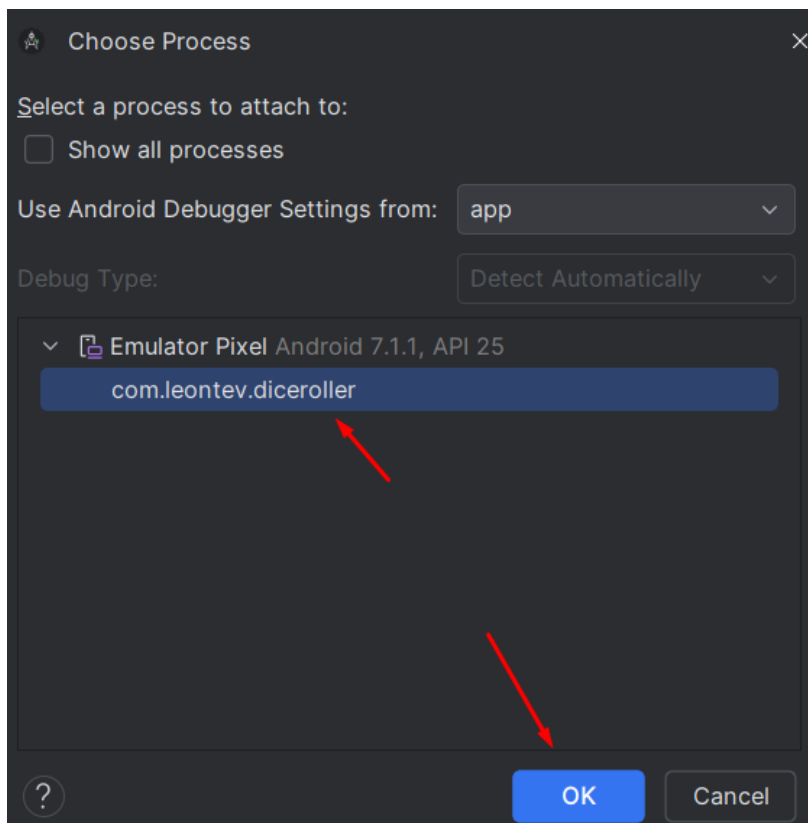
Чтобы присоединить отладчик к процессу приложения, выполните следующие действия **(!при уже запущенном приложении!)**:

1. Нажмите **Attach Debugger to Android Process**.

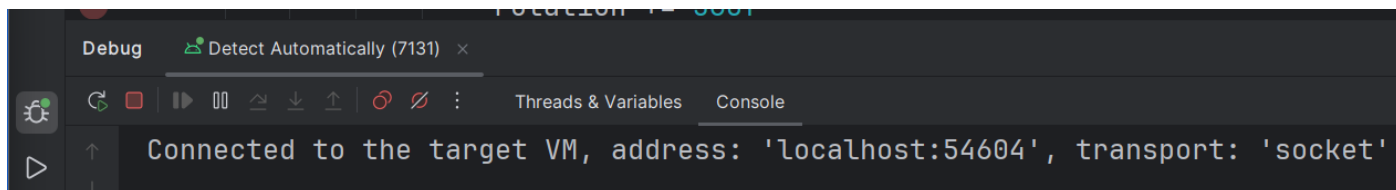


Откроется диалоговое окно «**Choose Process**», в котором вы можете выбрать процесс, к которому хотите присоединить отладчик.

2. Выберите *com.leontev.diceroller* и нажмите **OK**.



В нижней части **Android Studio** появится панель **отладки** с сообщением о том, что отладчик подключен к целевому устройству или эмулятору.

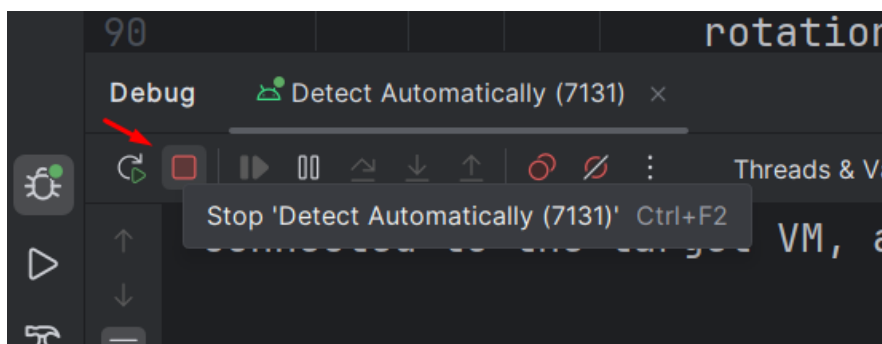


### Запустите приложение с помощью отладчика

Если вы с самого начала знаете, что хотите использовать отладчик, вы можете сэкономить время, запустив приложение с отладчиком. Более того, если вы хотите отладить код, который выполняется только при запуске приложения, вам необходимо запустить приложение с уже подключенным отладчиком.

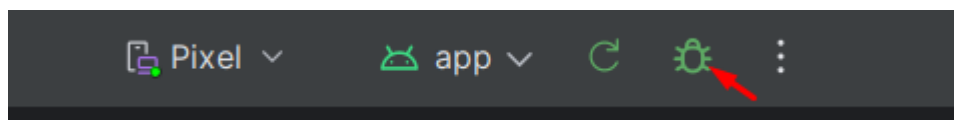
Чтобы запустить приложение с отладчиком, выполните следующие действия:

1. На панели «**Debug**» нажмите «**Stop**», а затем закройте приложение на устройстве или эмуляторе.





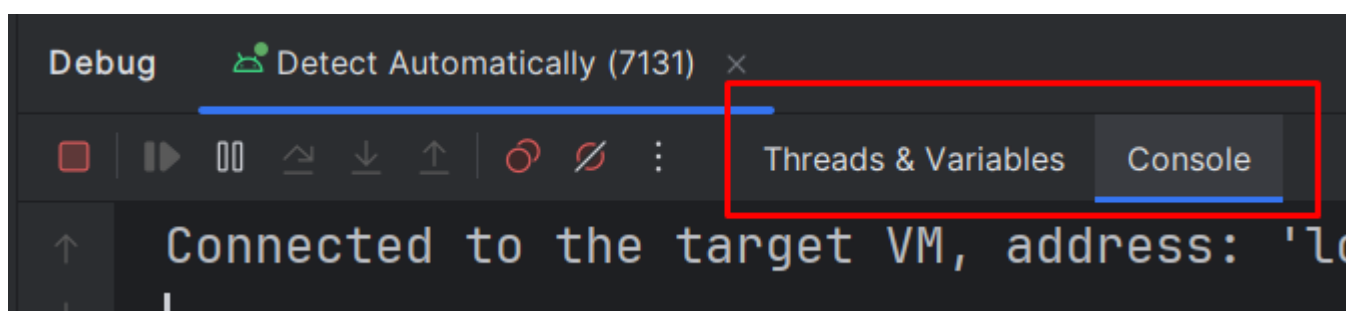
2. Нажмите «Отладка приложения» .



Та же панель **отладки** появляется в нижней части Android Studio с некоторым выводом консоли.

### Панель отладки

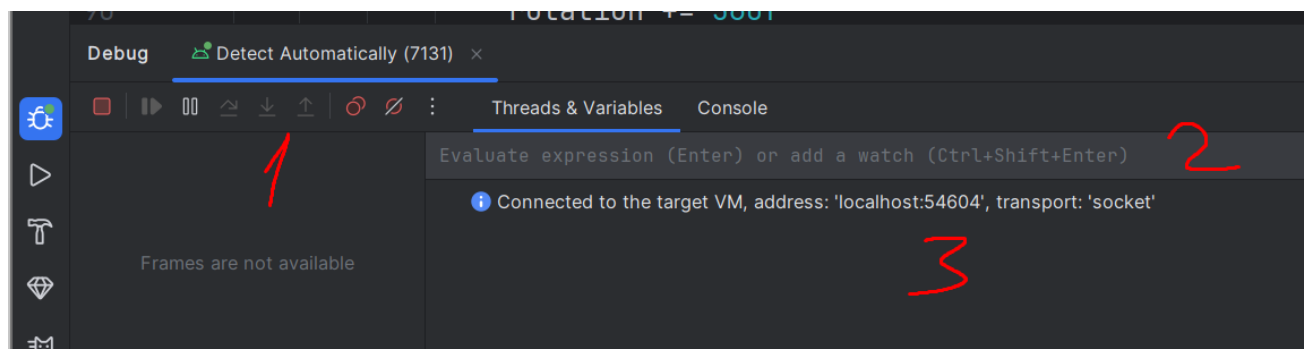
При первом запуске отладчика на панели «**Debug**» отображается ряд кнопок . В верхней части панели расположены кнопки «**Threads & Variables**» и «**Console**».



Кнопка «**Console**» отображает вывод **logcat** приложения. Если в вашем коде есть операторы **logcat**, вывод отображается по мере выполнения этого фрагмента кода.

Кнопка «**Debugger**» отображает три отдельные панели, которые в данный момент пусты, поскольку вы не используете отладчик:

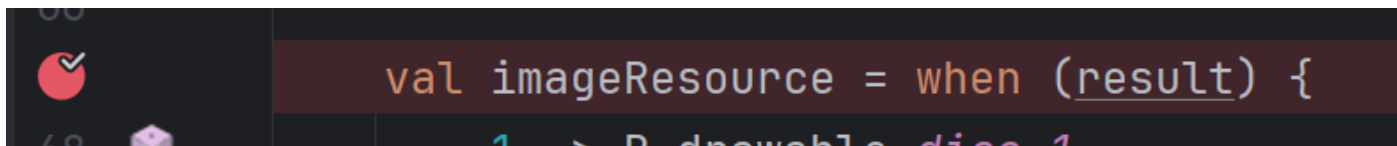
1. Отображение кадров
2. Оценка и наблюдение за записью выражения
3. Панель переменных



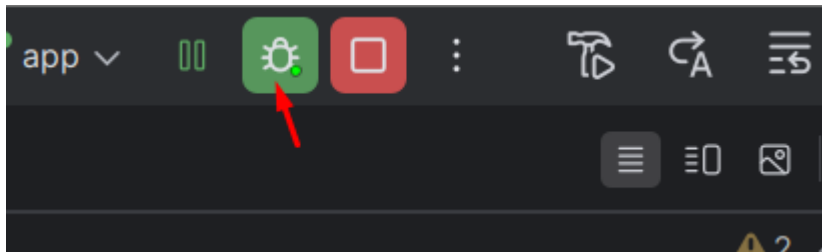
### Установка точки останова

Одной из основных функций отладчика является возможность остановки выполнения определенной строки кода с помощью точки останова.

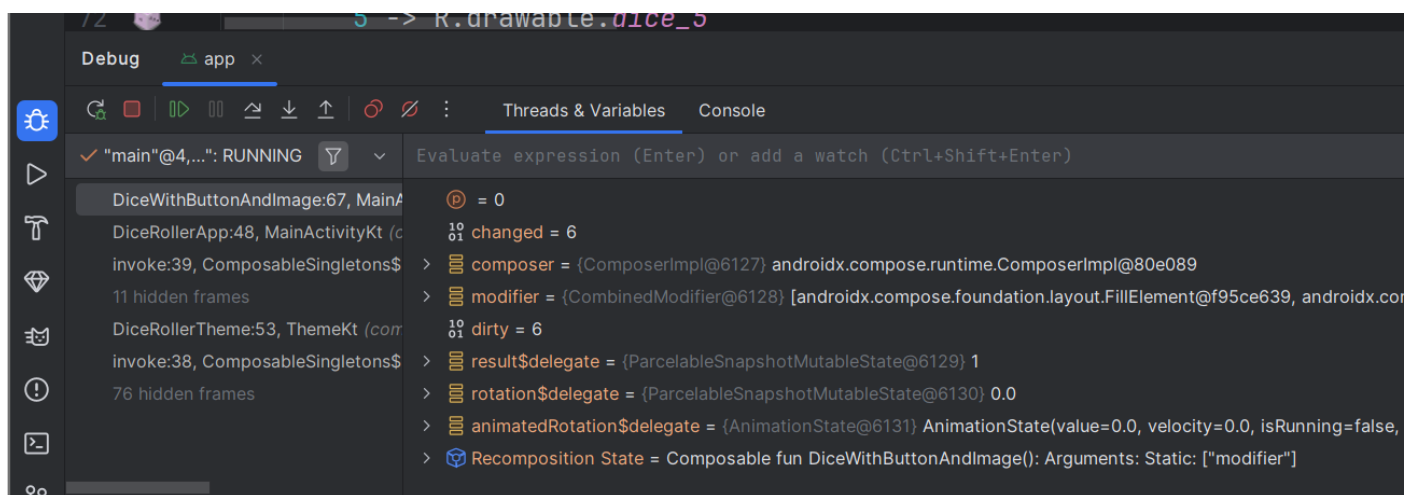
Установите точку останова у переменной **imageResource**:



Снова запустите дебаггер:



После запуска приложения вы увидите примерно такое изображение:



- На панели «**Кадры**» отображается большой объем информации, включая выделенную ссылку на строку, где была установлена точка останова.
- На панели «**Переменные**» отображается ряд элементов, но в этом приложении переменных немного, поэтому в рамках данной лабораторной работы информация не так уж важна. Однако возможность проверки переменных — важная функция отладчика, поскольку она позволяет понять, что происходит в коде во время выполнения. В этой лабораторной работе проверка переменных будет подробно рассмотрена далее.

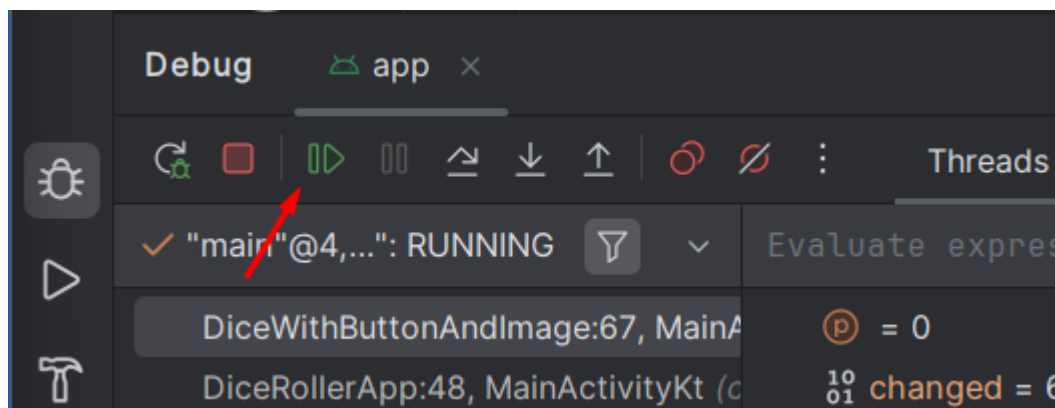
Если вы посмотрите на приложение на своём устройстве или эмуляторе, вы заметите, что экран пустой, поскольку приложение приостановлено на строке кода. Точнее, выполнение остановилось на точке останова, и пользовательский интерфейс ещё не отрисовался.

Имейте в виду, что приложение не всегда останавливается немедленно только из-за установки точки останова. Это зависит от того, где в коде установлена точка

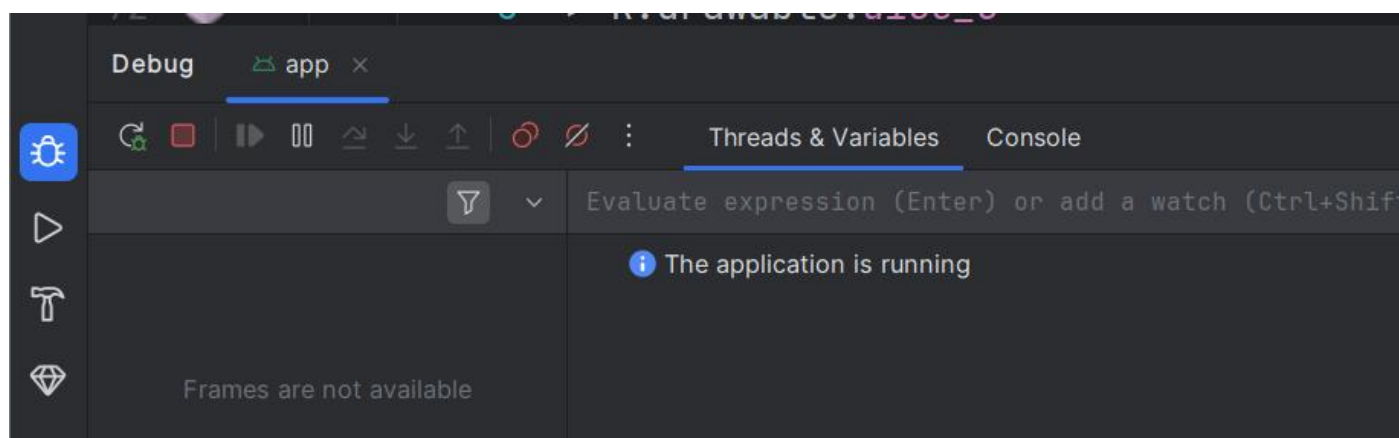
останова. В данном случае она была установлена на строке, которая выполняется при запуске приложения.

Важно помнить, что приложение приостанавливается на точке останова только при попытке выполнить строку, на которой она установлена. Существует несколько способов продолжить отладку, но сейчас вы можете использовать кнопку **«Resume Program»** .

Нажмите  **«Resume Program»** .



Теперь вы должны увидеть:



Большая часть информации исчезает, а кнопки снова становятся неактивными. Приложение также отображается на вашем устройстве или эмуляторе в обычном режиме. Причина в том, что код больше не приостановлен в точке останова, и приложение находится в нормальном состоянии выполнения. Отладчик подключен, но он не выполняет никаких действий до тех пор, пока не будет предпринята попытка выполнить строку кода с установленной точкой останова. Оставьте эту точку останова, так как она пригодится в следующих примерах.

### Кнопка **«Step Into»**

Кнопка «**Шаг с заходом**» отладчика — удобный способ углубить анализ кода во время выполнения. Если инструкция вызывает метод или другой фрагмент кода, кнопка «**Шаг с заходом**» позволяет вам войти в код, не перемещаясь туда вручную перед запуском отладчика для установки точки останова.

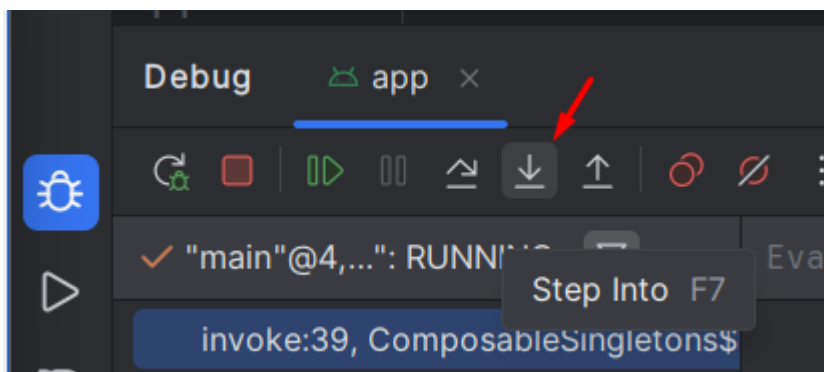
Чтобы использовать кнопку «**Step Into**», выполните следующие действия:

1. Создайте точку останова в теле лямбда-выражения **setContent** в функции **onCreate()** класса **MainActivity**, в котором вызывается функция **DiceRollerApp()**.

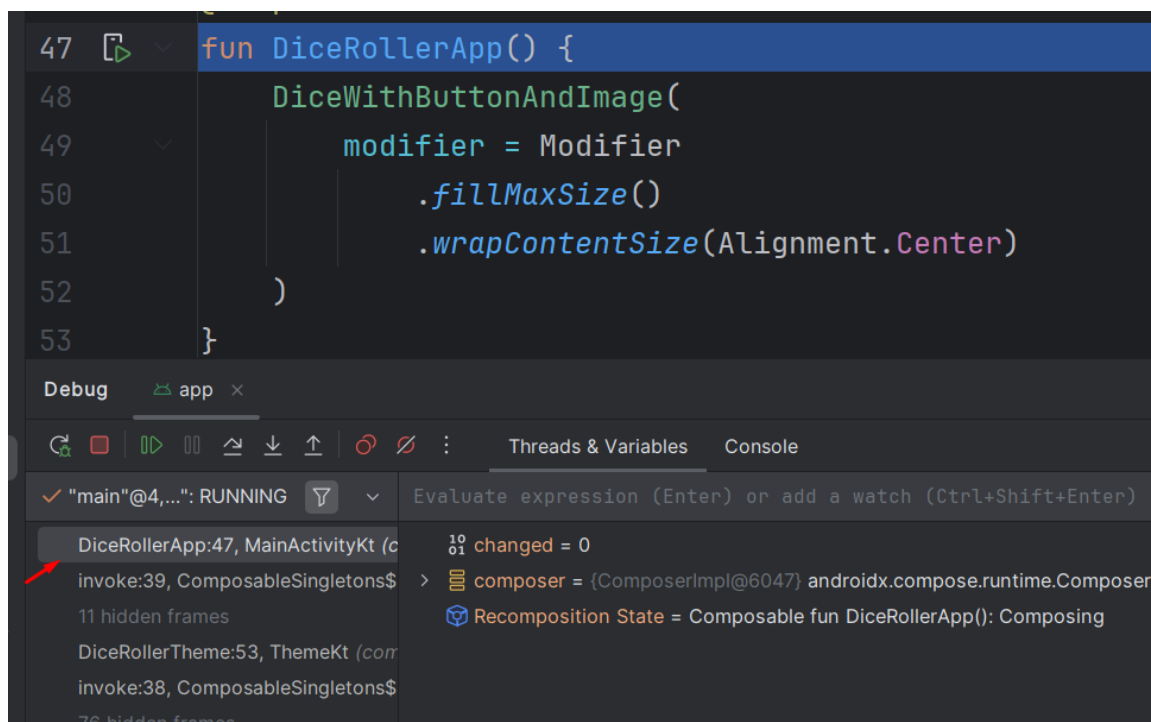


2. Нажмите  «**Отладка приложения**», чтобы перезапустить приложение с помощью отладчика. Выполнение приостанавливается на строке вызова функции DiceRollerApp().

3. Нажмите  «**Step Into**».



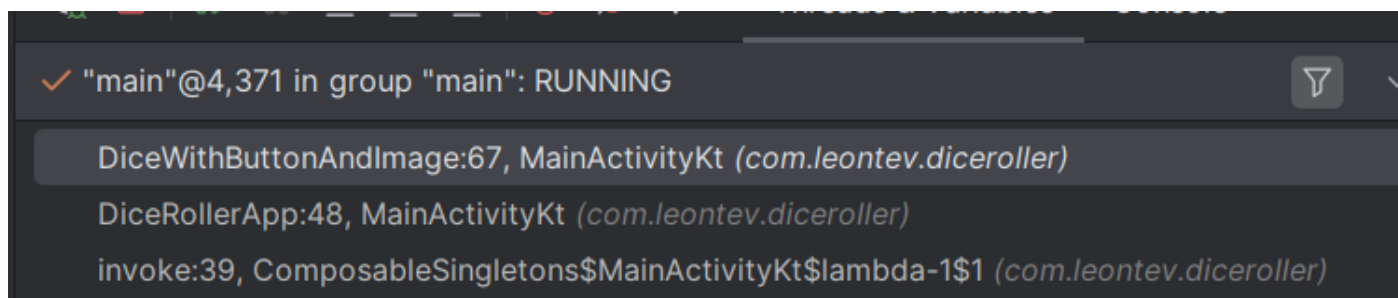
Теперь строка 47 выделена, а панель «**Кадры**» на панели «**Отладка**» показывает, что код приостановлен на строке 47.



4. Нажмите  «Возобновить программу» .

Это должно привести вас к исходной точке останова, которую вы установили.

В стеке вызовов видно, что функция `DiceWithButtonAndImage()` была приостановлена на строке 67, а функция была вызвана из строки 48 в функции `DiceRollerApp()`, которая была вызвана на строке 39.



Кнопка «Шаг зайти» позволяет войти в функцию и приостановить её выполнение, не устанавливая точку останова в самой функции. В этом случае точка останова устанавливается на вызове функции `DiceRollerApp()`. При нажатии кнопки «Шаг зайти» выполнение приостанавливается в `DiceRollerApp()` функции.

Dice Roller — довольно небольшое приложение, поскольку в нём не так много файлов, классов и функций. При работе с большими приложениями функция отладчика «Step Into» становится ещё более полезной, поскольку она позволяет детально изучить код, не перемещаясь по нему самостоятельно.

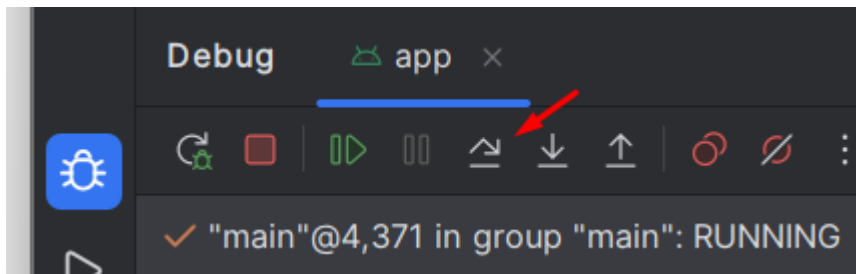
**Кнопка «Step Over»**



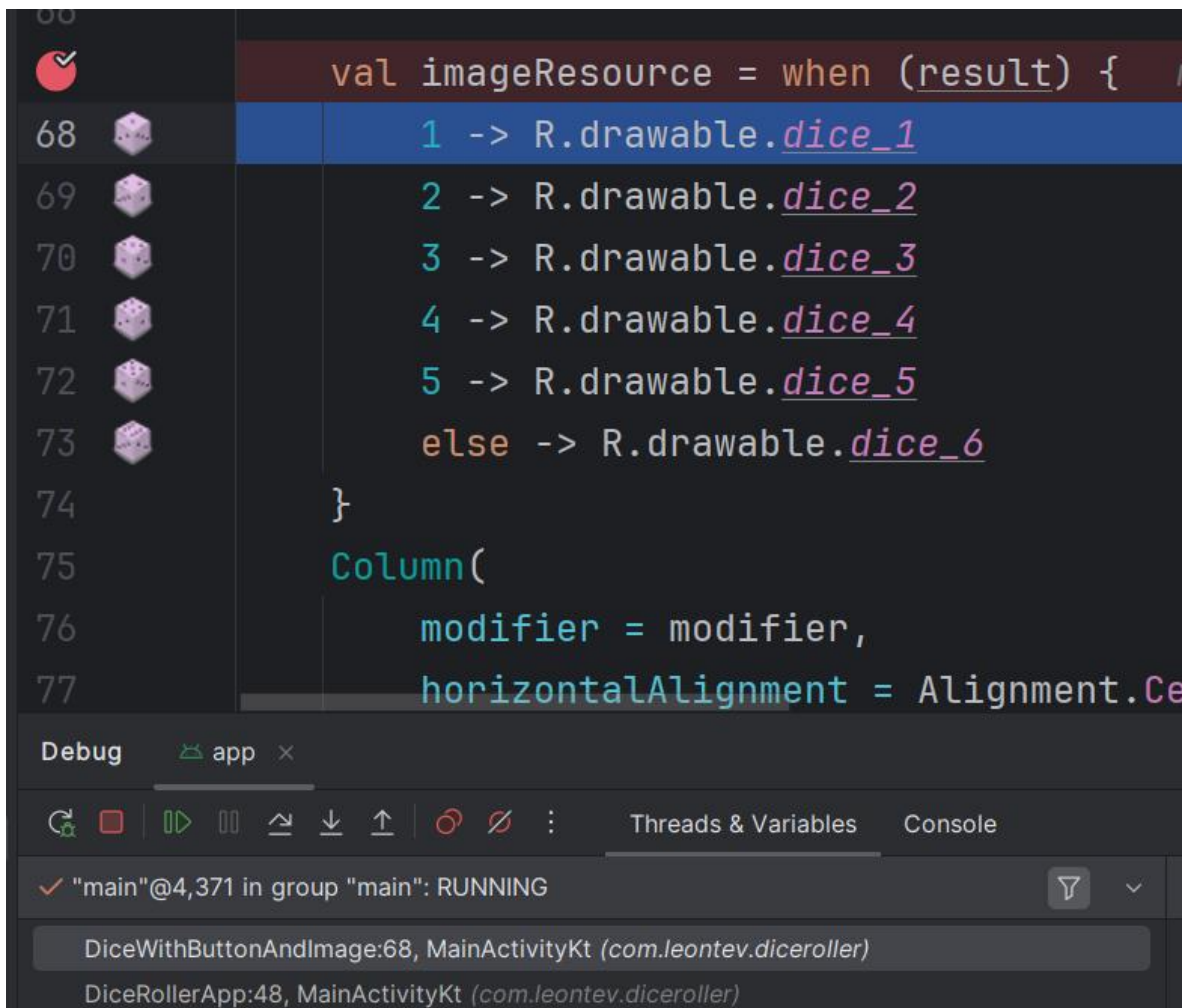
Кнопка «**Step Over**» предоставляет ещё один способ пошагового выполнения кода приложения во время выполнения. Она переводит выполнение на следующую строку кода и запускает отладчик.

Чтобы использовать кнопку «**Шаг за шагом**», выполните следующие действия:

- Нажмите  «Перешагнуть».



Теперь вы видите, что отладчик приостановил выполнение кода на следующей строке выполнения, а именно на строке 68. Вы можете продолжить пошагово выполнять каждую строку последовательно.

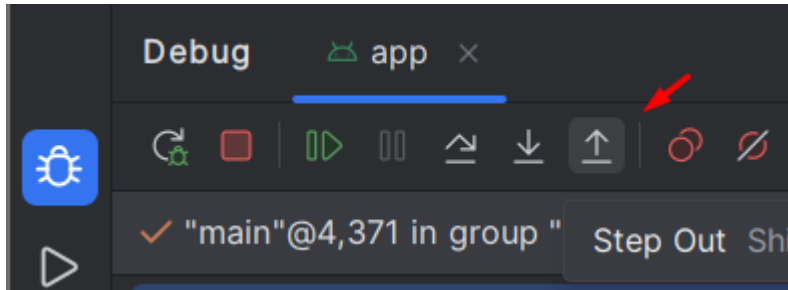


**Кнопка «Step Out»**



Кнопка «**Шаг наружу**» действует противоположно кнопке «**Шаг внутрь**». Вместо того, чтобы углубляться в стек вызовов, кнопка «**Шаг наружу**» перемещается по нему вверх.

1. Нажмите  «**Step Out**».



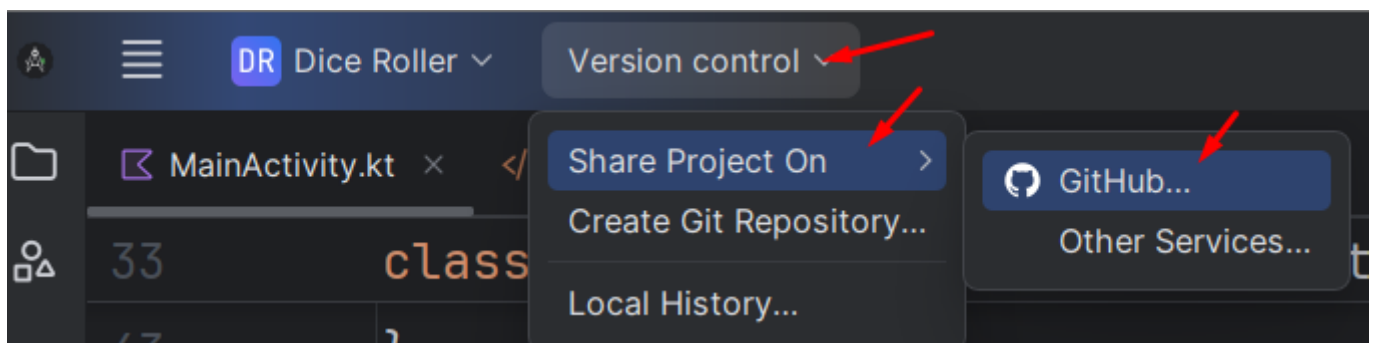
2. Обратите внимание, что отладчик вышел из функции `DiceRollerApp()` и вернулся к строке, которая ее вызвала.

Кнопка «**Step Out**» — полезный инструмент, когда вы зашли слишком далеко в стек вызовов методов. Она позволяет вам перемещаться по стеку вызовов, не выполняя пошагово весь код каждого метода, в который вы вошли.

## Шаг 10. Подключение проекта к GitHub

Теперь подключим ваш проект к **Git** и загрузим его на **GitHub**:

1. Перейдите в меню **Version Control** → **Share Project on** → **GitHub**.



3. Укажите имя репозитория (по умолчанию будет имя проекта, без пробела).
4. Нажмите **Share**, чтобы отправить проект в облако:

